Document Number: P0379R0
Date: 2016-05-27
Audience: L(E)WG (P0206 didn't go through SG1)
Reply to: Detlef Vollmann

Why `joining_thread` from P0206 is a Bad Idea

Abstract

P0206R1 proposes a class for joining threads that join on destruction. This paper shows why such a thread is a dangerous facility and should not be standardized.

Motivation for joining threads

P0206R0 provides a single example for motivating joining threads:

```
std::vector<std::pair<unsigned int, unsigned int>> partitions =
  utils::partition_indexes(0, size-1, num_threads);
std::vector<std::thread> threads;

LOG(LOG_DEBUG, "controller::reload_all: starting reload threads...");
for (unsigned int i=0; i<num_threads-1; i++) {
  threads.push_back(std::thread(reloadrangethread(this,
  partitions[i].first, partitions[i].second, size, unattended)));
}

LOG(LOG_DEBUG, "controller::reload_all: starting my own reload...");
this->reload_range(partitions[num_threads-1].first,
  partitions[num_threads-1].second, size, unattended);

LOG(LOG_DEBUG, "controller::reload_all: joining other threads...");
for (size_t i=0; i<threads.size(); i++) {
  threads[i].join();
}
```

P0206R0 doesn't give any more background on this example, so let's look at a very similar example for which I have background.

```
std::vector<std::pair<unsigned int, unsigned int>> partitions =
    partitionIndexes(0, size-1, numThreads);
std::vector<std::thread> threads;

// starting computing threads...
for (unsigned int i=1; i<numThreads; i++) {
    threads.push_back(std::thread{computeRangeThread
                                  , this
                                  , partitions[i].first
                                  , partitions[i].second});
}

// starting my own computation...
this->computeRange(partitions[0].first,
                   partitions[0].second);

// joining other threads...
for (size_t i=0; i<threads.size(); i++) {
    threads[i].join();
}
```

The main difference to the original example (apart from some renaming and removing unused arguments) is the fact that the starting thread doesn't compute the last partition, but the first.

Problem with the example

The new example still has the same problem as the old one: it may terminate if `push_back` throws. This is a well known problem. P0206R1 proposes to introduce `std::joining_thread` that behaves like `std::thread` but joins on destruction if the thread is joinable

(instead of calling `terminate`).
If we modify the new example to use `joining_thread` the original problem is indeed gone: the program will not terminate if `push_back` throws an exception. Instead, it will deadlock!

The new problem cannot be understood from just looking at the given piece of code, some more knowledge about `computeRangeThread` is required (unlike the old problem that didn't require additional knowledge to be seen). The new example comes from a course on parallelism in C++ that shows how to compute prime numbers using the Sieve of Eratosthenes in parallel. For this purpose, it makes sense to first compute an initial set of prime numbers and then using only these numbers for a pool of parallel threads that each runs the sieve algorithm on a specific range.
For this, all the threads that are pushed into the vector have to wait until the initial set is ready before they can actually start. This is done inside `computeRangeThread` and `computeRange` using a latch. But if `push_back` throws, the initial set is never computed and thus the join in the destructor will wait forever: deadlock!

While the sieve example may look constructed to demonstrate the problem (in this case we could just run the initial partition before we start the others), some synchronization between threads is very common. Actually, this is exactly why people are using `std::thread` instead of just using `std::async`.

Deadlock vs. terminate

Having to decide between deadlock and terminate, the choice should be easy: `terminate()` is well defined (and can be customized using `set_terminate`), deadlock is just undefined behaviour. And the general rule is anyway: "If you have to fail, fail fast."
In many environments, a terminated process is just restarted automatically, while deadlock causes (possibly long) delays that may have safety and security impacts.

Solutions

Parallelism

The example above fails because is requires concurrent forward progress guarantees (see P0072R1). If you have tasks that don't require this kind of synchronization, they only require parallel forward progress guarantees. So use the tool that gives you exactly this: `std::async()`. `async` has a destructor that behaves well if `push_back()` and similar mechanisms fail with an exception.

Concurrency

`std::async()` shouldn't be used in the above (new) example. Even if `launch::async` gives concurrent forward progress guarantees, it will still deadlock if `push_back()` fails. Therefore `std::thread` is the right tool here, but some kind of wrapper guard is required similar to the one proposed as "Solution 3" in P206R0, but a wrapper that is **not** just joining, but provides a mechanism that fulfills the synchronization protocol in a way to avoid deadlocks.

`joining_thread`

`joining_thread` as proposed is a non-solution: it replaces one problem by another even more severe problem.

Conclusion

P0072 allows us to talk about the synchronization requirements of our tasks. And we have (even now) in C++ mechanisms that accommodate the different synchronization requirements: `std::thread` for concurrent forward progress guarantees and `std::async` for parallel forward progress guarantees.
`joining_thread` would break this clear and easy to teach mapping: it looks like a thread and provides concurrent forward progress guarantees, but may cause deadlock when used with tasks that actually need this guarantee, thus defeating programmers, code reviewers, maintainers and teachers of C++ in their quest to produce safe, secure and understandable programs.